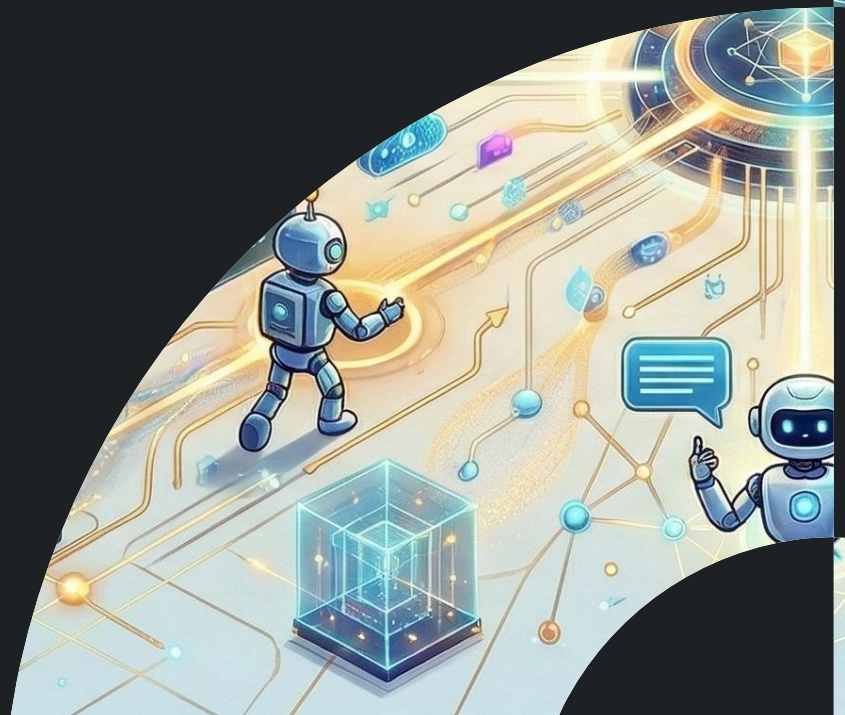
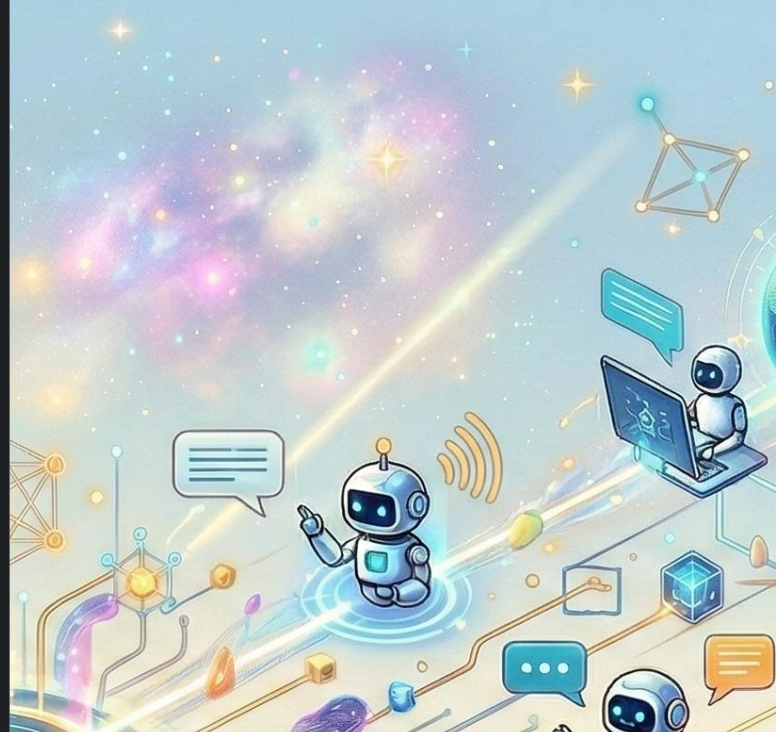
WebMCP: Welcome to the Agentic Web

Flutter Meetup March 2026





onepoint.
beyond the obvious

$ whoami _

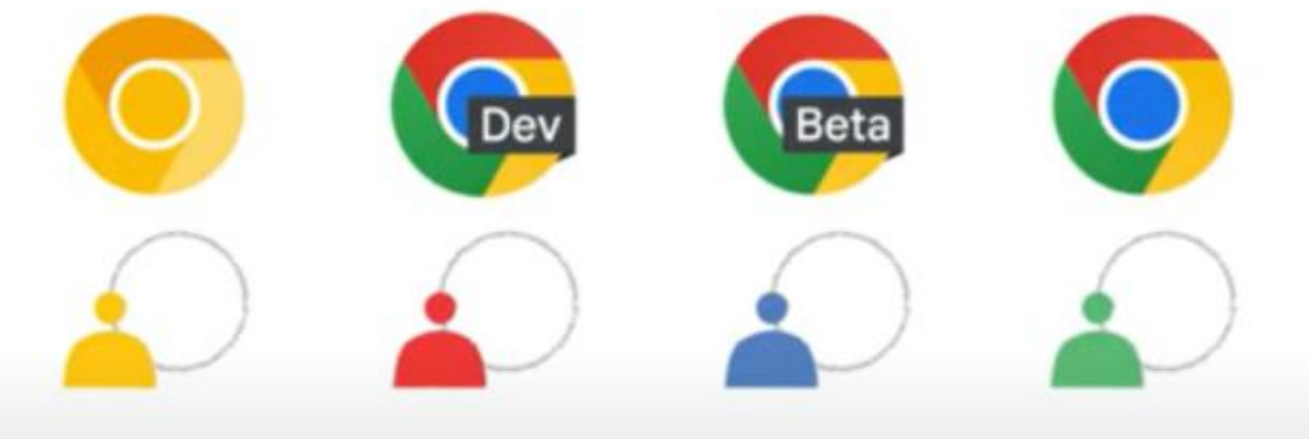
- **Lester Botello**
- Development Team Lead @ Wepoint
- 20+ years developing software, specially mobile and web.
- @lesterbotello everywhere.

Agenda

1. WebMCP explained
2. Use cases
3. API breakdown
4. Integration with Flutter apps
5. Demo
6. Limitations
7. Conclusion

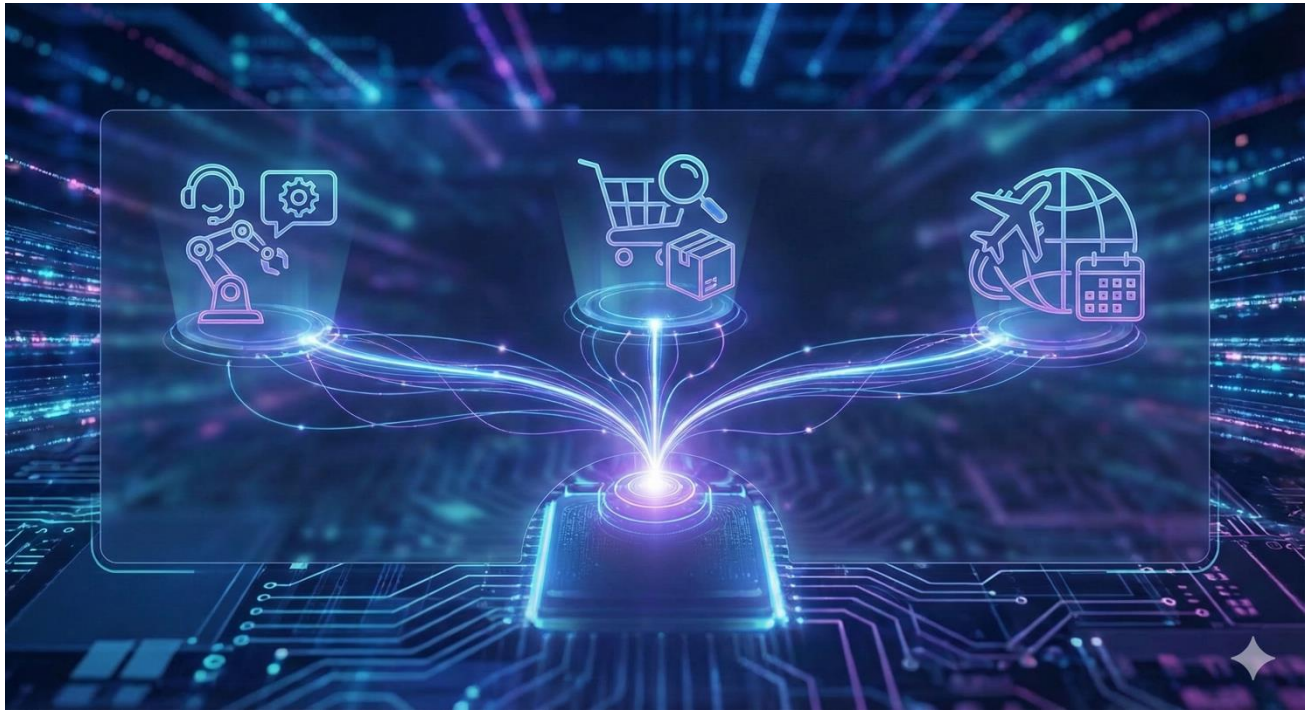
What is WebMCP?

- Proposed standard aiming to add agentic tool capabilities to ANY website
- Compatible with any DOM or JavaScript capable website (including apps made with Flutter Web!)
- Should work side-by-side with DOM actuation, instead of replacing it
- Currently in early preview *(requires Chrome Canary 146.0.7672.0 or higher)*
  - Can sign in to the early preview program in the Chrome for Developers website



Real-world usage examples

- **Customer support:** Detailed support ticket creation and smart routing/assignment.
- **E-commerce:** Agentic product search and shopping, better checkout and shipping.
- **Travel:** Flight search and booking.



Declarative API

- Defines standard actions using HTML forms.
- Will be ignored by non-compliant web browsers or browsers that explicitly disable agentic interaction.

```
<form toolname="addTodo" tooldescription="Add a new item to the todo list" action="/submit">

  <label for="text">Category:</label>
  <input type=text name=text>

  <select name="select" required toolparamtitle="Category"
                                 toolparamdescription="The category of the TODO">
    <option value="Option 1">Work</option>
    <option value="Option 2">Reminders</option>
    <option value="Option 3">Entertainment</option>
  </select>

  <button type=submit>Submit</button>
</form>
```

Imperative API

- Used to perform more complex and dynamic interactions using standard JavaScript.
- Non-compliant web browsers or browsers that explicitly disable agentic interaction will report called APIs as non-existent.

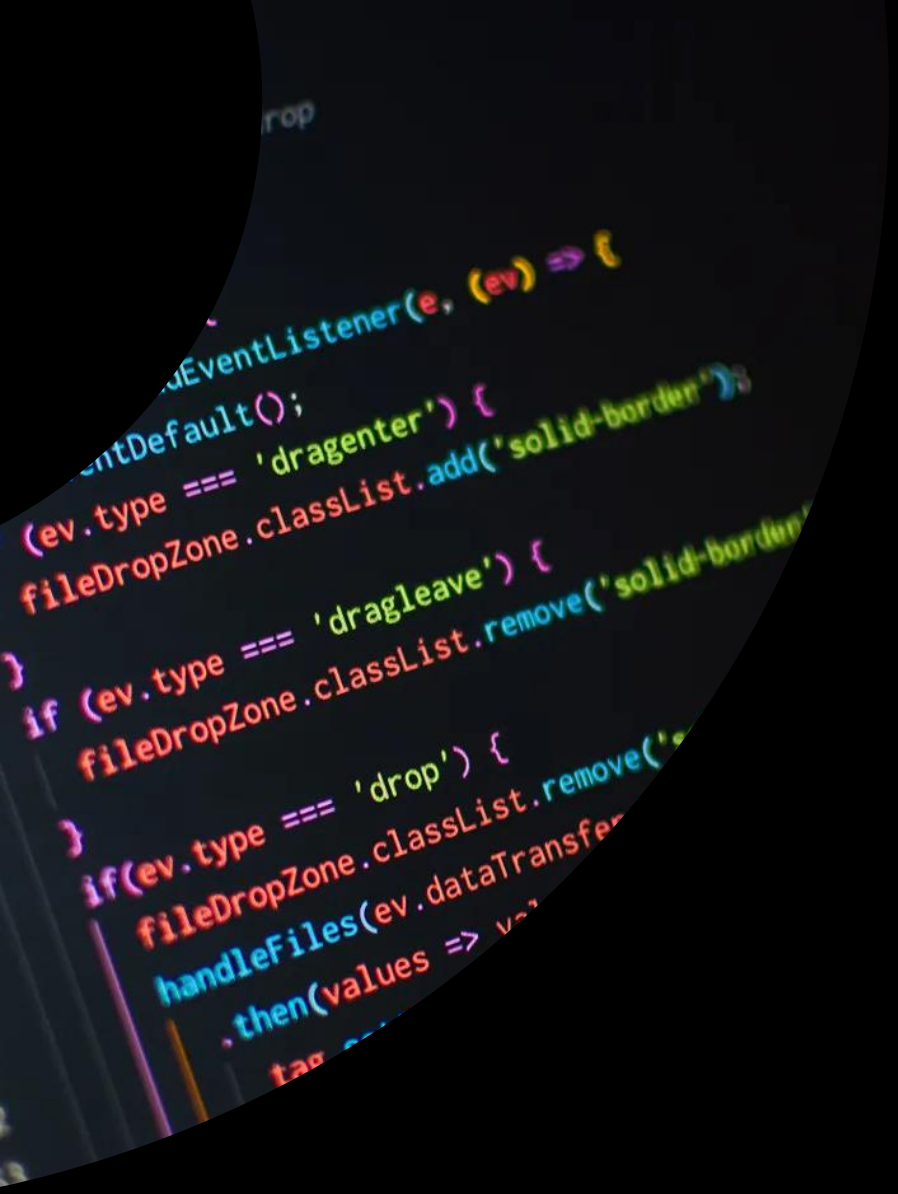
```javascript
if (window.navigator && window.navigator.modelContext) {
    window.navigator.modelContext.registerTool({
        name: "addTodo",
        description: "Add a new item to the todo list",
        inputSchema: {
            type: "object",
            properties: {
                title: { type: "string" },
                dueDateIso: { type: "string" },
                projectId: { type: "string" }
            }
        },
        annotations: {
            readOnlyHint: "true"
        },
        execute: ({ title, dueDateIso, projectId }) => {
            // Execute the addTodo function in the Flutter app
            window.addTodo(title, dueDateIso, projectId);
            return { content: [{ type: "text", text: `Added todo: ${title}` }] };
        }
    });
}
```

Leveraging JS Interop

- *JS Interop* makes it very easy to add WebMCP tools to your Flutter Web app.
- It's possible to expose Dart code to JavaScript for execution.

```
import 'dart:js_interop';
import 'dart:js_interop_unsafe';

void exportAddTodo(
  void Function(String title, [String? dueDateIso, String? projectId]) addTodo,
) {
  globalContext.setProperty('addTodo'.toJS, addTodo.toJS);
}
```


(ev.type === 'dragenter') {
fileDropZone.classList.add('solid-border');
if (ev.type === 'dragleave') {
fileDropZone.classList.remove('solid-border
if(ev.type === 'drop') {
fileDropZone.classList.remove(
handleFiles(ev.dataTransfe
.then(values =>

Demo

What can't be done with WebMCP?

- Browsing context is required.
- Because of this, there's no built-in discovery mechanism.
- Devs must ensure UI synchronization
- Complex UI may require refactoring in order to make sense of tools.

Thank you!